

Corso: Linguaggi Formali e Compilatori

GeoScript

Manuale d'uso

Guida completa e approfondita alla programmazione di grafica vettoriale



Studenti:

Cesari Matteo [Mat. 1073570]

Girolamo Davide [Mat. 1073645]

Indice dei Contenuti

Introduzione	4
Cos'è GeoScript: Visione e Scopo	4
La Filosofia del Linguaggio	4
Prerequisiti e Destinatari	4
Installazione e Avvio dell'Applicazione	5
Requisiti di Sistema	5
Procedura di Installazione.....	5
Lancio dell'Applicazione	5
Fondamenta del Codice	6
Creazione di uno Script.....	6
Il Sistema di Coordinate.....	6
L'importanza dei Commenti e della Leggibilità	7
Definizione dell'Ambiente e Configurazione.....	8
Il comando CANVAS.....	8
Gestione Avanzata dei Colori (DEF): Costanti e Temi.....	8
Logica dei Dati: Variabili ed Espressioni	9
Variabili Mutabili (VAR).....	9
Operatori Matematici e Calcolo Dinamico	9
Espressioni Parametriche nelle Coordinate	10
Primitive Grafiche	11
Forme Rettangolari	11
Forme Curvilinee	11
Tracciati Lineari	11
Poligoni e Geometria Complessa.....	12
Annotazioni e Tipografia.....	12
Stile, Attributi e Gerarchia Visiva	13
Il Colore in GeoScript.....	13
Ordine di Disegno	13
Controllo del Flusso: L'Intelligenza del Codice	14
Logica Condizionale (IF/THEN/ELSE).....	14

Cicli e Iterazioni (WHILE)	14
Iterazioni Determinate (FOR)	15
<i>Esempi Avanzati e Casi d'Uso</i>	17
Generazione Procedurale: "Il Tunnel"	17
Visualizzazione Dati: Grafico a Barre Parametrico	18
Modern Art	19

Introduzione

Cos'è GeoScript: Visione e Scopo

GeoScript è molto più di un semplice strumento di disegno; è un linguaggio di programmazione **Domain-Specific (DSL)** creato specificamente per colmare il divario tra codice e rappresentazione visiva. Progettato per la generazione rapida di grafica vettoriale (SVG), GeoScript offre un'alternativa potente ai software di grafica tradizionali basati su interfaccia ("point-and-click") e alla codifica manuale di file **SVG**, che può risultare verbosa e incline a errori.

Mentre un artista tradizionale usa pennello e tela, un utente GeoScript usa logica e matematica. Questo approccio permette di creare non solo immagini statiche, ma "ricette" visive che possono generare infinite variazioni cambiando pochi parametri iniziali. È ideale per la creazione di pattern complessi, frattali geometrici, layout architettonici semplificati e grafici basati sui dati.

La Filosofia del Linguaggio

GeoScript adotta un paradigma **imperativo e strutturato**. Questo significa che ogni riga di codice è un ordine diretto ed esplicito impartito a un "pittore virtuale".

La filosofia alla base della sintassi è la **leggibilità**. Comandi come `LINE FROM (0,0) TO (100,100)` sono auto-esplicativi, leggibili quasi come una frase in lingua inglese. Tuttavia, sotto questa semplicità sintattica si nasconde la potenza di un vero linguaggio di programmazione, completo di:

- **Astrazione:** Uso di variabili per non ripetere valori.
- **Controllo:** Strutture decisionali (IF) per alterare il disegno in base ai dati.
- **Automazione:** Cicli (FOR, WHILE) per eseguire operazioni ripetitive migliaia di volte senza fatica.

Prerequisiti e Destinatari

GeoScript è stato progettato per essere accessibile. Non è richiesta una laurea in informatica o anni di esperienza in computer graphics.

- **Per il principiante:** È un ottimo modo per imparare i concetti base della programmazione (variabili, loop, coordinate) con un feedback visivo immediato.
- **Per l'esperto:** Offre un modo rapido per prototipare visualizzazioni o generare asset grafici programmatici senza il peso di librerie grafiche complesse.

Una conoscenza elementare di geometria cartesiana (asse X e Y) è l'unico vero requisito tecnico.

Installazione e Avvio dell'Applicazione

Requisiti di Sistema

Per eseguire GeoScript, il sistema deve soddisfare i seguenti requisiti:

- **Java Development Kit (JDK):** Versione 8 o superiore.
- **Librerie Esterne:** antlr-runtime.jar (necessaria per interpretare la grammatica).
- **Spazio su Disco:** Minimo (meno di 10MB per i file compilati).

Procedura di Installazione

GeoScript non richiede un installer grafico; è un'applicazione basata su Java.

1. **Scarica il progetto:** Assicurati di avere la cartella con i file .class (o il file .jar se generato) e la cartella delle librerie.
2. **Configura il Classpath:** Assicurati che il file antlr-runtime.jar sia incluso nel percorso delle librerie del tuo progetto.

Lancio dell'Applicazione

L'applicazione viene lanciata da riga di comando (Terminale o CMD). Il compilatore legge un file di testo (es. test.geo) e genera il file output.svg.

Comando tipico per la compilazione (se non usi un IDE come Eclipse o IntelliJ):

```
java -cp ".;lib/antlr-runtime.jar" myCompilerPackage.Main test.geo
```

Fondamenta del Codice

Creazione di uno Script

Un file sorgente GeoScript (con estensione .geo) viene interpretato sequenzialmente, dall'alto verso il basso. La punteggiatura è cruciale: ogni singola istruzione deve terminare obbligatoriamente con un punto e virgola ;. Questo segnala al compilatore che il comando è concluso.

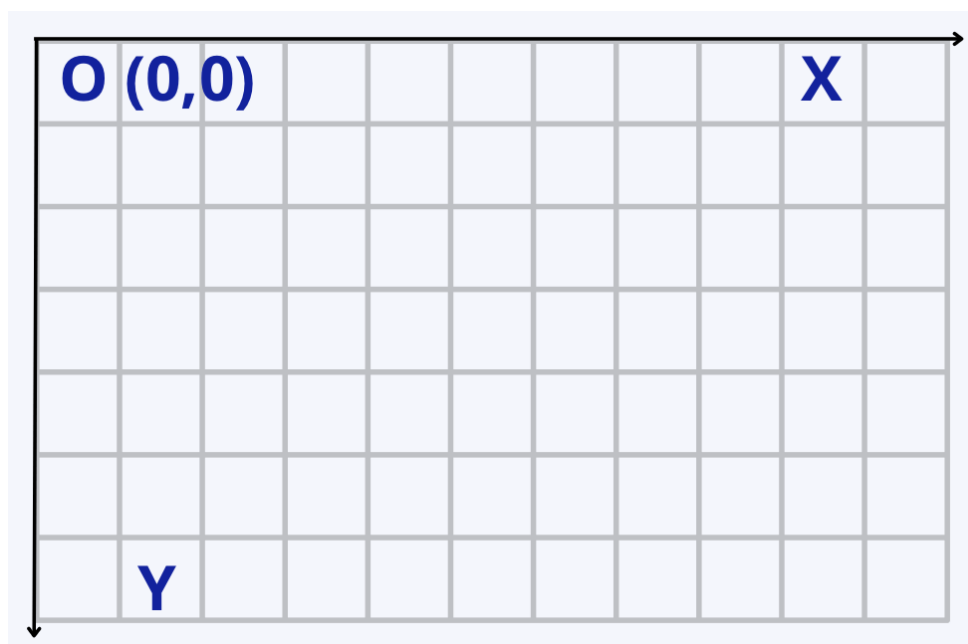
Un programma ben strutturato segue generalmente questo flusso logico, che garantisce stabilità e manutenibilità:

1. **Setup dell'Ambiente:** Definizione delle dimensioni della tela (CANVAS).
2. **Definizione delle Costanti:** Assegnazione di nomi ai colori (DEF) per creare una palette coerente.
3. **Stato Iniziale:** Dichiarazione e inizializzazione delle variabili (VAR) che verranno usate per calcoli o posizionamenti.
4. **Logica e Rendering:** Il cuore del programma, dove avvengono i calcoli e i comandi di disegno effettivi.

Il Sistema di Coordinate

GeoScript utilizza il sistema di coordinate standard per la grafica computerizzata (raster e vettoriale), che differisce leggermente da quello appreso in matematica alle scuole superiori.

- **Origine (0, 0):** Si trova nell'angolo in **alto a sinistra** della tela.
- **Asse X (Orizzontale):** I valori positivi crescono verso **destra**.
- **Asse Y (Verticale):** I valori positivi crescono verso il **basso**.



Implicazioni pratiche:

- Un punto (100, 200) è spostato di 100 pixel a destra e 200 pixel in basso rispetto all'origine.
- Aumentare la Y significa "scendere" nella pagina.
- Coordinate negative (es. -50, -50) sono tecnicamente valide ma disegneranno fuori dall'area visibile della tela (off-canvas).

L'importanza dei Commenti e della Leggibilità

Scrivere codice che funziona è solo metà del lavoro; scrivere codice leggibile è l'altra metà. GeoScript supporta commenti che vengono ignorati dal compilatore ma sono preziosi per l'essere umano.

- **Commenti di linea (//):** Utili per brevi annotazioni a margine.
- **Commenti a blocco (/* ... */):** Ideali per descrivere sezioni complesse o per disabilitare temporaneamente blocchi di codice durante il debugging.

Esempio di Buona Pratica:

```
// --- Sezione Sfondo ---  
/* Disegno un rettangolo grande quanto il canvas  
   per simulare un colore di background */  
RECT AT (0,0) SIZE (800,600) FILL #EEEEEE;
```

Definizione dell'Ambiente e Configurazione

Il comando CANVAS

Il comando CANVAS definisce i confini del tuo grafico. Sebbene il compilatore possa avere dei default, definire esplicitamente il canvas è fondamentale per garantire che le proporzioni del tuo disegno siano corrette su qualsiasi dispositivo.

Sintassi:

```
CANVAS(larghezza, altezza);
```

Esempio:

```
CANVAS(1920, 1080);
```

Imposta una tela in risoluzione Full HD. Tutte le coordinate successive dovrebbero idealmente rientrare in questi limiti (0-1920 per X, 0-1080 per Y).

Gestione Avanzata dei Colori (DEF): Costanti e Temi

Invece di disseminare codici esadecimali criptici (es. #FF5733) in tutto il codice, GeoScript incoraggia l'uso di definizioni semantiche tramite DEF. Questo crea delle costanti che non cambiano durante l'esecuzione.

Vantaggi dell'uso di DEF:

- **Leggibilità:** fill rosso_fuoco è più chiaro di fill #FF4500.
- **Manutenibilità (Temi):** Se decidi di cambiare il "rosso" in "blu" per tutto il disegno, devi modificare solo una riga all'inizio del file, invece di cercare e sostituire centinaia di valori.

Sintassi:

```
DEF nome_costante = #RRGGBB;
```

Esempio di Palette:

```
DEF primary_color = #3498DB;    // Blu
DEF accent_color = #E74C3C;     // Rosso
DEF text_color = #2C3E50;       // Grigio scuro
```


Logica dei Dati: Variabili ed Espressioni

La vera potenza di GeoScript emerge quando si smette di usare numeri fissi e si inizia a usare la matematica dinamica.

Variabili Mutabili (VAR)

A differenza di DEF, che definisce valori immutabili (colori), VAR definisce contenitori numerici il cui valore può cambiare nel tempo. Le variabili rappresentano lo "stato" del tuo programma: contatori di cicli, posizioni correnti, dimensioni calcolate.

Sintassi:

```
VAR nome_variabile = espressione;
```

Esempio Dinamico:

```
VAR step = 50;  
VAR posizione_x = 0;  
// Ora posso incrementare posizione_x in un ciclo  
posizione_x = posizione_x + step;
```

Operatori Matematici e Calcolo Dinamico

GeoScript supporta le quattro operazioni fondamentali, permettendo di calcolare posizioni e dimensioni "al volo".

- “+” (Somma)
- “-” (Sottrazione)
- “*” (Moltiplicazione)
- “/” (Divisione intera: attenzione, $5 / 2$ restituisce 2, non 2.5)

L'uso delle parentesi () è fondamentale per controllare l'ordine delle operazioni, proprio come in algebra. È stato anche implementato il supporto all'operatore Modulo “%”.

Espressioni Parametriche nelle Coordinate

Puoi inserire intere espressioni matematiche ovunque il linguaggio si aspetti un numero (coordinate, dimensioni, raggi).

Esempio di Design Parametrico:

Immagina di voler disegnare un quadrato sempre perfettamente centrato, indipendentemente dalla larghezza del canvas.

```
DEF canvas_w = 800;  
DEF canvas_h = 600;  
VAR lato_quadrato = 200;  
  
// Calcolo dinamico del centro: (LarghezzaTotale-LarghezzaOggetto)/2  
VAR pos_x = (canvas_w - lato_quadrato) / 2;  
VAR pos_y = (canvas_h - lato_quadrato) / 2;  
  
SQUARE AT (pos_x, pos_y) SIZE lato_quadrato;
```

Se si modifica lato_quadrato a 300, il quadrato rimarrà perfettamente centrato senza ricalcolare le coordinate manualmente.

Primitive Grafiche

GeoScript offre un set di primitive vettoriali ottimizzate. Comprendere come viene definito il "punto di ancoraggio" di ogni forma è vitale per il posizionamento preciso.

Forme Rettangolari

RECT (Rettangolo)

La forma più comune. Il punto di ancoraggio AT (x,y) è l'angolo in alto a sinistra.

Sintassi: RECT AT (x, y) SIZE (w, h) [Stile];

SQUARE (Quadrato)

Forma sintattica per un rettangolo equilatero. Richiede un solo parametro per la dimensione.

Sintassi: SQUARE AT (x, y) SIZE lato [Stile];

Forme Curvilinee

CIRCLE (Cerchio)

A differenza dei rettangoli, i cerchi sono definiti a partire dal loro centro.

Sintassi: CIRCLE AT (cx, cy) RADIUS r [Stile];

Nota: Disegnare un cerchio a (0,0) ne mostrerà solo un quarto (il settore in basso a destra), poiché il resto sarà fuori canvas (coordinate negative).

ELLIPSE (Ellisse)

Come il cerchio, ma con due raggi distinti per definire schiacciamento orizzontale o verticale.

Sintassi: ELLIPSE AT (cx, cy) RADII (rx, ry) [Stile];

Tracciati Lineari

LINE (Linea)

Il collegamento più breve tra due punti. Le linee sono uniche perché non hanno un concetto di "interno", quindi l'attributo FILL viene ignorato; conta solo STROKE.

Sintassi: LINE FROM (x1, y1) TO (x2, y2) STROKE colore;

Poligoni e Geometria Complessa

TRIANGLE (Triangolo)

Definisce un triangolo tramite tre vertici espliciti. Sebbene richieda un parametro AT per coerenza sintattica, la forma effettiva è determinata dai punti in POINTS.

Sintassi: TRIANGLE AT (x, y) POINTS (x1, y1, x2, y2, x3, y3) [Stile];

Consiglio: Usare AT (0,0) come convenzione se si definiscono coordinate assolute in POINTS.

POLYGON (Poligono Libero)

Lo strumento più flessibile. Collega una serie arbitraria di punti nell'ordine dato e chiude automaticamente la forma (connettendo l'ultimo punto al primo).

Sintassi: POLYGON POINTS (x1, y1, x2, y2, x3, y3, ...) [Stile];

Uso: Ideale per stelle, esagoni o forme irregolari.

Annotazioni e Tipografia

TEXT (Testo)

Essenziale per etichettare grafici o aggiungere titoli.

Sintassi: TEXT ("Messaggio", x, y) COLOR colore;

Nota: Le coordinate (x,y) rappresentano solitamente l'angolo in basso a sinistra della prima lettera (baseline).

Stile, Attributi e Gerarchia Visiva

Senza stile, le forme sarebbero invisibili o uniformi. GeoScript permette di controllare l'aspetto visivo tramite attributi opzionali.

Il Colore in GeoScript

Ogni forma possiede due proprietà fondamentali:

- **FILL (Riempimento):** Il colore interno dell'area della forma.
- **STROKE (Bordo):** La linea di contorno della forma.

Se un attributo viene omesso, il comportamento dipende dall'implementazione del renderer (spesso il default è trasparente per il fill e nero per lo stroke, o viceversa). Specificare esplicitamente i colori è sempre la scelta migliore.

Sintassi Combinata:

```
RECT AT (10,10) SIZE (50,50) FILL #FF0000 STROKE #000000;
```

Ordine di Disegno

GeoScript disegna le forme nell'esatto ordine in cui compaiono nel codice.

- Le forme scritte **prima** (in alto nel file) saranno sullo **sfondo**.
- Le forme scritte **dopo** (in basso nel file) saranno in **primo piano**, coprendo quelle precedenti in caso di sovrapposizione.

Conseguenza: Disegnare sempre lo sfondo (RECT grande quanto il canvas) come primissima istruzione del programma.

Controllo del Flusso: L'Intelligenza del Codice

Qui è dove GeoScript passa dall'essere un elenco di istruzioni a un vero programma intelligente.

Logica Condizionale (IF/THEN/ELSE)

Permette allo script di prendere decisioni basate sui dati. Ad esempio, colorare una barra di un grafico di rosso se il valore è troppo basso, o di verde se è alto.

Operatori di confronto:

- “<” Minore
- “>” Maggiore
- “==” Uguale (Nota: doppio uguale per il confronto)
- “!=” Diverso

Schema:

```
IF (variabile > soglia) THEN {  
    // Fai questo se vero  
} ELSE {  
    // Fai questo se falso  
}
```

Cicli e Iterazioni (WHILE)

Il ciclo WHILE è potente ma pericoloso. Ripete il blocco di codice fintanto che la condizione è vera.

Schema:

```
WHILE (condizione) {  
    // Blocco di codice da ripetere  
    // fintanto che la condizione è VERA (diversa da 0)  
}
```

Attenzione ai Cicli Infiniti: È vitale assicurarsi che all'interno del blocco { ... } ci sia un'istruzione che modifica la variabile controllata (es. $i = i + 1$), altrimenti il ciclo non terminerà mai e il programma si bloccherà.

Iterazioni Determinate (FOR)

Il ciclo FOR è la struttura prediletta per creare griglie, pattern ripetitivi o serie di elementi. Condensa in una sola riga: inizializzazione, condizione e incremento.

Schema:

```
FOR (init; condition; increment) {  
    // Body  
}
```

Esempio pratico (Griglia Orizzontale):

```
VAR i = 0;  
// Disegna 10 rettangoli distanziati di 60 pixel  
FOR (i = 0; i < 10; i = i + 1) {  
    RECT AT (i * 60, 100) SIZE (50, 50);  
}
```

Trasformazioni Globali

Le trasformazioni permettono di alterare l'intero sistema di coordinate, invece di spostare le singole forme.

- **TRANSLATE (x, y):** Sposta l'origine (0,0) in una nuova posizione. Tutto ciò che viene disegnato dopo questo comando risulterà spostato.

Esempio:

```
CANVAS(400, 400);  
  
// Sposto l'origine al centro del canvas  
  
TRANSLATE(200, 200);  
  
// Ora il punto (0,0) è il centro del foglio.  
// Questo cerchio apparirà esattamente al centro.  
  
CIRCLE AT (0, 0) RADIUS 50 FILL #FF5733;
```

- **ROTATE (gradi):** La rotazione avviene sempre attorno all'origine (0,0). Se vuoi ruotare un oggetto su se stesso, di solito devi prima usare un TRANSLATE.

Esempio:

```
CANVAS(500, 500);  
  
TRANSLATE(100, 100);  
  
ROTATE(45); // Ruota tutto di 45 gradi in senso orario  
  
// Questo rettangolo apparirà inclinato  
  
RECT AT (0, 0) SIZE (100, 50) FILL #33FF57;
```

- **SCALE (sx, sy):** Utile per creare versioni più piccole o più grandi dello stesso disegno senza cambiare i numeri delle dimensioni.

Esempio:

```
CANVAS(400, 400);  
  
// Disegno originale  
  
SQUARE AT (10, 10) SIZE 20 FILL #3357FF;  
  
SCALE(2, 2); // Raddoppia le dimensioni di tutto ciò che segue  
// Questo quadrato, pur avendo SIZE 20, apparirà grande 40  
  
SQUARE AT (40, 10) SIZE 20 FILL #FF33F1;
```


Esempi Avanzati e Casi d'Uso

Generazione Procedurale: "Il Tunnel"

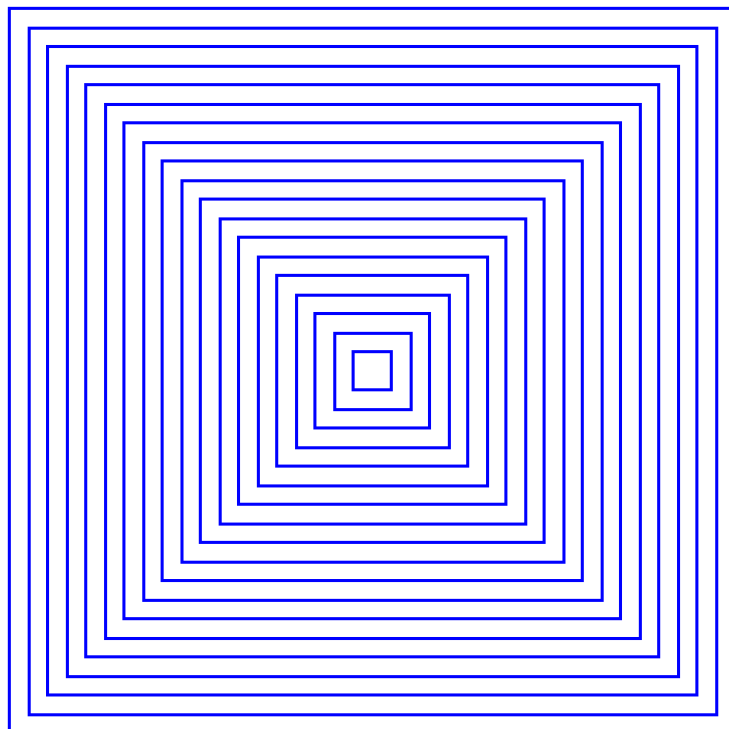
Questo esempio mostra come creare un effetto di profondità visiva usando un semplice ciclo e matematica sottrattiva.

```
CANVAS(500, 500);
DEF blu = #0000FF;
VAR i = 0;
VAR size = 0;

// 20 iterazioni per creare 20 cornici concentriche
FOR (i = 0; i < 20; i = i + 1) {
  // A ogni passo, la dimensione diminuisce
  size = 500 - (i * 25);

  // Calcoliamo l'offset per mantenere il quadrato centrato:
  // (DimensioneTotale - DimensioneAttuale) / 2
  VAR pos = (500 - size) / 2;

  // Disegna solo il contorno per vedere attraverso
  SQUARE AT (pos, pos) SIZE size STROKE blu;
}
```



Visualizzazione Dati: Grafico a Barre Parametrico

GeoScript può essere usato per generare grafici basati sui dati. Ecco come disegnare tre barre con altezze basate su variabili.

```
CANVAS(400, 300);
DEF bar_color = #2ECC71;

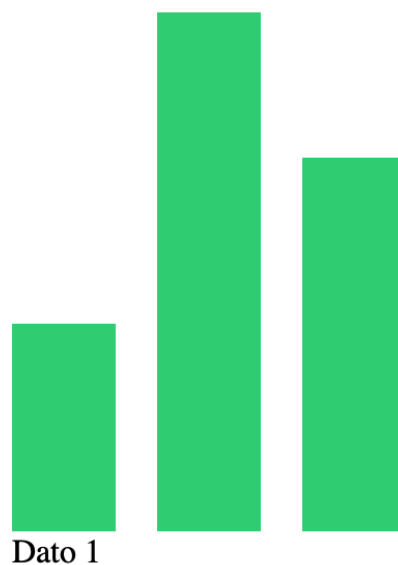
// I dati (Input)
VAR val1 = 100;
VAR val2 = 250;
VAR val3 = 180;

// Configurazione Grafico
VAR larghezza_barra = 50;
VAR gap = 20;
VAR base_y = 280; // Linea di base in basso

// Barra 1
RECT AT (50, base_y - val1) SIZE (larghezza_barra, val1) FILL
bar_color;
TEXT("Dato 1", 50, 295) COLOR #000000;

// Barra 2 (spostata a destra)
RECT AT (50 + larghezza_barra + gap, base_y - val2)
    SIZE (larghezza_barra, val2) FILL bar_color;

// Barra 3
RECT AT (50 + (larghezza_barra + gap) * 2, base_y - val3)
    SIZE (larghezza_barra, val3) FILL bar_color;
```



Modern Art

GeoScript puo essere anche utilizzato per comporre motivi geometrici anche con senso estetico.

```
CANVAS(800, 600);

DEF bg_color = #ECF0F1;      // Grigio molto chiaro
DEF color_dark = #2C3E50;    // Blu scuro (Midnight Blue)
DEF color_teal = #1ABC9C;    // Turchese
DEF color_orange = #E67E22;  // Arancione
DEF color_red = #E74C3C;     // Rosso
DEF color_white = #FFFFFF;

RECT AT (0, 0) SIZE (800, 600) FILL bg_color STROKE bg_color;

VAR rows = 12;
VAR cols = 16;
VAR size = 40;
VAR gap = 50;
VAR startX = 25;
VAR startY = 25;
VAR r = 0;
VAR c = 0;

FOR (r = 0; r < rows; r = r + 1) {
  // Ciclo per le colonne
  FOR (c = 0; c < cols; c = c + 1) {
    VAR posX = startX + c * gap;
    VAR posY = startY + r * gap;
    VAR sum = r + c;
    VAR mod = sum - (sum / 2) * 2;
    IF (mod == 0) THEN {
      // Quadrati scuri
    }
  }
}
```

```

        SQUARE AT (posX, posY) SIZE size FILL color_dark STROKE
color_white;
    } ELSE {
        // Quadrati turchesi
        SQUARE AT (posX, posY) SIZE size FILL color_teal STROKE
color_white;
    }
}
}
VAR centerX = 400;
VAR centerY = 300;
VAR maxRadius = 160;
VAR step = 20;
VAR currentRadius = maxRadius;
VAR iter = 0;
WHILE (currentRadius > 0) {
VAR mod_iter = iter - (iter / 2) * 2;
IF (mod_iter == 0) THEN {
    CIRCLE AT (centerX, centerY) RADIUS currentRadius FILL
color_orange STROKE color_white;
} ELSE {
    CIRCLE AT (centerX, centerY) RADIUS currentRadius FILL color_red
STROKE color_white;
}
currentRadius = currentRadius - step;
iter = iter + 1;

}

```

```

TRIANGLE AT (0, 0) POINTS (0, 0, 150, 0, 0, 150) FILL color_dark
STROKE color_white;

TRIANGLE AT (800, 600) POINTS (800, 600, 650, 600, 800, 450) FILL
color_dark STROKE color_white;

LINE FROM (0, 300) TO (800, 300) STROKE color_white;

LINE FROM (400, 0) TO (400, 600) STROKE color_white;

RECT AT (250, 540) SIZE (300, 40) FILL color_white STROKE
color_dark;

TEXT("GeoScript Modern Art", 290, 565) COLOR color_dark;

```

